iCodeGuru × DigiTech Transformation

AI Engineering Bootcamp

Lecture Structure

Each week follows the same structured

Day	Type	Purpose	Format
Monday	Theory	Core concepts, LLM internals, architecture deep-dives	Lecture
Tuesday	Practical	Hands-on lab applying Monday's concepts	Guided coding lab
Wednesday	Theory	Next topic introduced, builds on Monday	Lecture
Thursday	Practical	Hands-on lab applying Wednesday's concepts	Guided coding lab
Friday	Revision	Full-week recap	Discussion

4-Week Overview

Week	Theme	Mon/Wed (Theory)	Tue/Thu (Practical)	Friday (Revision)	Deliverable
W1	LLM Foundations + Prompting	LLM anatomy, APIs, Prompt Engineering, Structured Outputs	SDK coding labs, Streamlit app build	Recap	AI Writing Coach (Streamlit)
W2	RAG Pipelines	Embeddings, Vector DBs, RAG from scratch, Advanced retrieval	RAG lab, Hybrid search, RAGAS eval	Recap	Document Intelligence Chatbot
W3	Agents + Multi-Agent	ReAct, LangGraph, Tool use, Agentic RAG	Agent build lab, multi-agent system	Recap	Autonomous Research Agent
W4	Production + Deployment	FastAPI, Docker, Evaluation, Observability, Security	API build lab, containerization	Final recap	Production AI API Service

WEEK 1 - LLM Foundations + Prompting + First App

Goal	Understand how LLMs work at a deeper level and confidently build with them
Deliverable	AI Writing Coach - live Streamlit app with JSON-mode tone analysis + token sidebar
Tools	OpenAI SDK, Anthropic SDK, Streamlit, Ollama, tiktoken

Weekly Schedule

Day	Type	Topic	Deliverable
Monday	Theory	LLM Internals: Transformers, tokenisation, RLHF, instruction tuning	Lecture
Tuesday	Practical	Lab: Call OpenAI + Claude APIs, count tokens, compare outputs	Guided coding lab
Wednesday	Theory	Prompt Engineering: Zero-shot, few-shot, CoT, system prompts, structured outputs	Lecture
Thursday	Practical	Lab: Build AI Writing Coach - personas, JSON-mode tone analysis, Streamlit UI	Guided coding lab
Friday	Revision	Recap: LLM anatomy + prompting patterns quiz, peer review of apps, Q&A	Reflection

Monday - LLM Internals (Theory)

- How transformers work: attention mechanism, why it matters for AI engineers
- Tokenisation deep dive: BPE, vocabulary, why token count affects cost + quality
- Training pipeline: pre-training → RLHF → instruction tuning - what each stage produces
- Context windows: hard limits, what happens when you exceed them, strategies to work within them
- Temperature, top-p, top-k: when to tune these and why
- Model selection: GPT-4.1 mini vs Claude Haiku 4.5 vs Claude Sonnet 4.6 vs local Ollama - cost vs quality trade-offs.

Wednesday - Prompt Engineering + Structured Outputs (Theory)

- Prompt anatomy: role, task, format, constraints, examples - each element's function
- Zero-shot, few-shot, chain-of-thought: when each pattern applies
- System prompts: persona design, boundary setting, tone control
- Structured outputs: JSON mode, function calling, Pydantic output parsers
- Prompt injection basics + defence strategies

- Cost optimisation: token budgeting, model routing, caching identical calls
- Context window overflow: what happens when context is exceeded, three mitigations - summarise older turns, sliding window truncation, use a larger-context model
- Multimodal inputs - awareness only

Tuesday - API Lab (Practical)

- Set up OpenAI SDK and Anthropic SDK in a clean environment
- Make API calls with different temperature and top-p values - observe output variation
- Count tokens with tiktoken - predict cost before running a query
- Run the same prompt on GPT-4.1 mini, Claude Haiku 4.5, and Ollama (Llama 3.2 3B) - compare outputs. Hardware note: Ollama with Llama 3.2 3B requires approximately 4GB of RAM and runs on most student machines. Students without Ollama access can compare GPT-4.1 mini vs Claude Haiku 4.5 only - the comparison exercise is equally valid with two cloud models.
- Extract structured JSON from an unstructured passage using function calling
- Multimodal lab extension: pass one image (any JPEG) to GPT-4o mini and Claude Haiku 4.5, print the description from each - observe that the same image input works across both APIs with minimal code change
- Wrap every API call in try/except catch APIError, RateLimitError, and timeout - print a clear message for each; this pattern is used in every lab from this point forward

Thursday - AI Writing Coach Build (Practical)

- Build Streamlit UI: dropdowns for tone/style, text area input, formatted output
- Add system prompt personas (academic, casual, professional)
- Implement JSON-mode tone analysis that returns scores for clarity, formality, readability
- Add a token usage sidebar showing prompt tokens, completion tokens, cost estimate
- Deploy to Streamlit Cloud - shareable link
- Add error handling to the app: catch API errors and display a user-friendly message in the UI rather than crashing - include a retry button

Friday - Week 1 Revision

- Recap: transformer architecture → tokenisation → prompt design → structured outputs
- Mini quiz: 10 questions on token counting, prompt patterns, model selection
- Peer review: swap apps, give structured feedback on prompt design decisions
- Discussion: where did the model fail? hallucinations observed? why?
- Preview Week 2: what problem does RAG solve that prompting alone can't?

Case Studies

- **Case Study 1A - AI Customer Support at Scale**
  - SaaS company: 10,000 tickets/month, 60% repetitive, goal: 8 hrs → 5 mins response
  - Deliverable: triage system prompt + escalation flowchart
- **Case Study 1B - AI Content Personalisation Engine**
  - EdTech: personalise quiz questions for 50,000 students using profile parameters
  - Deliverable: mini question generator with student profile parameter

WEEK 2 - RAG Pipelines + Document Intelligence

Goal	Build production-quality RAG pipelines that go beyond naive retrieval
Deliverable	Document Intelligence Chatbot - RAG app with citations, evaluation report, LangSmith tracing
Tools	LangChain, FAISS, ChromaDB, Pinecone, RAGAS, LangSmith, PyMuPDF. Pre-lab setup: create a LangSmith account and generate an API key before Tuesday's lab - students who skip this lose 15 minutes of lab time to account setup on the day. Pinecone Starter (free) tier is sufficient for the swap exercise: 5 indexes, 2GB storage, 2M write units/month, 1M read units/month - limited to the AWS us-east-1 region on serverless. Create your account before Thursday's lab.

Weekly Schedule

Day	Type	Topic	Deliverable
Monday	Theory	Embeddings, vector stores, RAG pipeline from scratch	Lecture
Tuesday	Practical	Lab: Build RAG from scratch over a PDF - chunking, FAISS, cosine similarity	Guided coding lab
Wednesday	Theory	Advanced retrieval: hybrid search, reranking, query expansion, HyDE, RAGAS eval	Lecture
Thursday	Practical	Lab: Upgrade to LangChain RAG with memory, citations, LangSmith tracing + RAGAS score	Guided coding lab
Friday	Revision	Recap	Reflection

Monday - Embeddings + Naive RAG from Scratch (Theory)

- What embeddings are: dense vectors, semantic meaning, distance in high-dimensional space
- Embedding models: OpenAI text-embedding-3-small vs Sentence Transformers (open-source)
- Chunking strategies: fixed-size, recursive, semantic - why chunk size and overlap matter
- Vector stores: FAISS (local), ChromaDB (local persistent), Pinecone (managed cloud)
- Building RAG from scratch: load → chunk → embed → store → retrieve → generate
- Cosine similarity vs dot product vs MMR - when to use each

iCodeGuru × DigiTech Transformation

Wednesday - Advanced Retrieval + Evaluation (Theory)

- Why naive RAG fails: ambiguous queries, multi-hop reasoning, irrelevant chunk retrieval
- Hybrid search: combining dense (semantic) + sparse (BM25) - almost always better than pure semantic
- Query expansion: generating multiple phrasings to improve recall
- HyDE (Hypothetical Document Embeddings): generate a fake answer, embed it, retrieve against it
- Reranking: cross-encoders, Cohere Rerank - re-score top-k results for precision
- RAGAS metrics: context precision, context recall, answer relevancy, faithfulness - what each measures
- LangSmith: tracing pipeline steps, debugging retrieval failures
- Conversational RAG: adding memory for multi-turn chat
- LlamaIndex awareness: what it is, when to choose it over LangChain - stronger multi-document reasoning and built-in document parsing abstractions; you will encounter both in production codebases
- Structured document parsing: real enterprise PDFs contain tables, multi-column layouts, and headers that PyMuPDF extracts poorly - Unstructured.io is the standard tool for these; mention LlamaParse and AWS Textract as alternatives
- Chunking for heterogeneous content: prose uses recursive splitter (already covered), tables should be kept whole as a single chunk, code blocks should never be split mid-function - content type determines chunking strategy

Tuesday - RAG from Scratch Lab (Practical)

- Load a PDF with PyMuPDF, chunk it with recursive splitter
- Generate embeddings with OpenAI text-embedding-3-small
- Store in FAISS, run semantic queries, inspect retrieved chunks
- Build a simple Q&A loop: query → retrieve → prompt → generate → print
- Experiment: change chunk size and overlap, observe retrieval quality differences

Thursday - Full RAG App Build (Practical)

- Rebuild the pipeline using LangChain components: loaders, splitters, retrievers, chains
- Add ChromaDB as a persistent store (survives restarts)
- Add conversational memory so the chatbot retains context across turns
- Add source citations to every response
- Connect LangSmith tracing to the pipeline and verify traces appear in the LangSmith dashboard

Friday - Week 2 Revision

- Recap: embeddings → chunking → vector search → advanced retrieval → evaluation
- Quiz: chunk size effects, hybrid search benefit, RAGAS metric definitions
- Peer compare: share RAGAS scores - whose pipeline scored highest and why?
- Discussion: where did retrieval fail in your testing? ambiguous queries? long documents?

- Preview Week 3: what can't RAG do that an agent can?

Case Studies

- **Case Study 2A - Legal Document Intelligence**
  - Search 100,000 contracts for auto-renewal clauses expiring this quarter
  - Deliverable: architecture diagram + 10-query evaluation plan
- **Case Study 2B - AI-Powered Internal Knowledge Base**
  - 500-person company losing $2M/year due to inaccessible Confluence/Slack/PDF knowledge
  - Deliverable: mini demo with 3 internal docs, present to class

WEEK 3 - Tool-Using Agents + Multi-Agent Systems

Goal	Build agents that plan, use tools, and collaborate - moving from passive Q&A to active systems
Deliverable	Autonomous Research Agent - LangGraph workflow with structured report output
Tools	LangGraph (primary), CrewAI (awareness), OpenAI Agents SDK, Tavily, Python REPL

Weekly Schedule

Day	Type	Topic	Deliverable
Monday	Theory	Agent architectures: ReAct, Plan-and-Execute; LangGraph fundamentals, tool use	Lecture
Tuesday	Practical	Lab: Build a ReAct agent with Tavily search + Python REPL tools in LangGraph	Guided coding lab
Wednesday	Theory	Agentic RAG, multi-agent patterns, agent memory, guardrails	Lecture
Thursday	Practical	Lab: Autonomous Research Agent - plan-search-read-synthesise-review workflow	Guided coding lab
Friday	Revision	Recap	Reflection

Monday - Agent Architectures + LangGraph (Theory)

- When to use agents vs RAG vs plain LLM calls - decision framework
- Agent architectures: ReAct (Reason-Act-Observe) and Plan-and-Execute - the two patterns built this week. Reflexion (self-critique loop) is referenced for awareness; it requires its own dedicated lab session and is not built here.
- LangGraph fundamentals: nodes, edges, shared state, conditional routing
- Tool use and function calling: defining tools LLMs can invoke
- Agent memory types: short-term (conversation), long-term (vector store), episodic, semantic
- Human-in-the-loop checkpoints: when and how to pause for human approval
- Safety basics: rate limiting, output validation, loop detection, infinite loop prevention

Wednesday - Advanced Agents + Multi-Agent Systems (Theory)

- Agentic RAG: combining retrieval with agent reasoning loops - when static RAG isn't enough
- Multi-agent patterns: supervisor + specialist, peer collaboration, CrewAI vs LangGraph

- MCP (Model Context Protocol): now an industry-standard protocol with 97M monthly SDK downloads and co-governed by Anthropic, OpenAI, Google, Microsoft, and AWS under the Linux Foundation (December 2025). Any MCP-compatible model can use any MCP server - one integration replaces per-tool custom code. A dedicated hands-on lab is included in Thursday's session.
- Agent evaluation: covered in Week 4 alongside production evaluation frameworks - how to test agentic systems is a production concern, not just a design concern, and fits better there
- Cost and latency of agentic loops: profiling token usage across multi-step workflows
- CrewAI overview: higher-level role-based abstraction - contrast with LangGraph's control
- Guardrails: input validation, output checking, toxic content filtering
- Agent communication protocol landscape - awareness only: MCP handles model-to-tool connections; Agent-to-Agent (A2A, published by Google in 2025) handles agent-to-agent communication; both are becoming standard infrastructure in multi-agent production systems
- MCP hands-on lab (30 minutes): define one MCP server exposing a single tool (e.g. a file reader or a weather lookup), connect it to the existing LangGraph agent, verify the agent calls it via the MCP protocol - students see the full tool-registration and invocation flow in practice

Tuesday - ReAct Agent Lab (Practical)

- Define Tavily web search and Python REPL as LangGraph tools
- Build a ReAct agent: Reason → Act → Observe cycle in a LangGraph state machine
- Add conditional routing: if tool returns error, retry or fall back to a different tool
- Add short-term memory so the agent retains context across tool calls
- Test with 5 research questions - observe where the agent succeeds and fails
- Write 3 pass/fail assertion tests on the agent's output: (1) response is not empty, (2) response contains at least one tool call in the trace, (3) response does not contain a known error string - run with pytest; this introduces the testing habit before Week 4 formalises evaluation

Thursday - Autonomous Research Agent (Practical)

- Build the full LangGraph workflow: Plan → Search → Read → Synthesise → Review
- Integrate Tavily for web search, web fetcher for full article text, text summariser
- Add long-term memory using a vector store for previously researched topics
- Implement structured report output using Pydantic models
- Profile the agent: count total tokens used, measure latency per step
- Record a 2-3 minute demo video showing the agent in action (homework – due before Friday's revision)

Friday - Week 3 Revision

- Recap: ReAct → LangGraph → Agentic RAG → multi-agent → guardrails
- Quiz: agent architecture selection, LangGraph concepts, when to use CrewAI vs LangGraph

- Demo sharing: play 3-4 student demo videos, class discussion on agent behaviour
- Discussion: what did the agent do unexpectedly? how would you add guardrails?
- Reminder: asyncio pre-reading assigned on Monday must be completed before Week 4 Monday. Check in with students - who has started? who needs help finding the resource?
- Preview Week 4: how do you take this from a demo to something real users can hit?

Case Studies

- **Case Study 3A - Autonomous Sales Development Agent**
  - Automate: prospect research → personalised email → CRM update
  - Deliverable: 3-step prototype with structured output at each stage
- **Case Study 3B - AI Code Review Agent**
  - AI-assisted code review for bugs, style, security before human review
  - Deliverable: agent that takes a Python function and returns structured feedback

WEEK 4 - Production, Deployment + Evaluation

Goal	Make AI apps production-ready: monitored, evaluated, secure, and deployable
Deliverable	Production AI API Service - FastAPI app with /chat, /index-document, /evaluate endpoints
Tools	FastAPI, Uvicorn, Docker, MLflow (or W&B - pick one), GitHub Actions, LangSmith

Weekly Schedule

Day	Type	Topic	Deliverable
Monday	Theory	FastAPI design, async LLM handlers, Pydantic models, secrets management	Lecture
Tuesday	Practical	Lab: Build /chat and /index-document endpoints, test with curl + Postman	Guided coding lab
Wednesday	Theory	Evaluation frameworks, LLM-as-judge, observability, Docker basics, CI with GitHub Actions	Lecture
Thursday	Practical	Lab: Add /evaluate endpoint, structured logging, Dockerfile, GitHub Actions workflow	Guided coding lab
Friday	Revision	Full program recap	Reflection

Monday - FastAPI + Production API Design (Theory)

- AI product lifecycle: prototype → evaluation → hardening → deployment → monitoring
- FastAPI: why it's the standard for AI services - async, type hints, auto docs
- Async LLM handlers: non-blocking calls so the server handles concurrent requests
- Pydantic models: request/response validation, why strict types prevent production bugs
- Secrets and environment management: .env files, never hardcode API keys
- API design patterns for AI: streaming responses, timeout handling, graceful degradation
- Security basics: prompt injection via API inputs, rate limiting, authentication

Wednesday - Evaluation, Observability + Docker (Theory)

- Evaluation frameworks: RAGAS, LangSmith, custom test suites - when to use each
- LLM-as-judge: using GPT-4.1 (or Claude Sonnet 4.6) to score your own app's outputs - prompt design for judges
- AI product metrics: latency per query, cost per query, hallucination rate, user satisfaction
- Structured logging: what to log (input, output, model, tokens, latency, errors) and why

- Observability vs monitoring: dashboards, alerts, drift detection
- LangFuse vs LangSmith: LangSmith is hosted and LangChain-native (used in Weeks 2-3); LangFuse is open-source, self-hostable, and framework-agnostic - it supports any model or framework and is the default choice for teams that need self-hosted tracing; both cover traces, evals, and prompt management
- OpenTelemetry for AI - awareness: OpenTelemetry became the official standard for AI/LLM observability in January 2025 when the AI/LLM semantic conventions were adopted; every model call becomes a traceable span with metadata (model, tokens, latency, cost); LangFuse and Datadog LLM Observability are both built on top of it
- Docker fundamentals: Dockerfile, image build, container run - containerising an AI app
- GitHub Actions: CI pipeline - lint, test, build image on every push

Tuesday - FastAPI Lab (Practical)

- Scaffold a FastAPI app: project structure, router files, middleware
- Build /chat endpoint: accept a user message, call the LLM, stream the response
- Build /index-document endpoint: accept a PDF, chunk + embed, store in ChromaDB
- Add Pydantic request/response models with validation
- Test with curl and Postman - document in README
- Add API key authentication header

Thursday - Evaluation + Containerisation (Practical)

- Add /evaluate endpoint: takes a question + ground truth answer, returns RAGAS scores
- Implement structured logging: log every request with input, output, model, latency, tokens, cost
- Harden the Dockerfile from Tuesday: add a non-root user, a health check endpoint, and a .dockerignore file to exclude secrets and notebooks
- Rebuild and verify the hardened image runs correctly - confirm health check passes and API key header is required
- Set up a GitHub Actions workflow: on push → lint with ruff → run tests → build Docker image
- Deploy to Hugging Face Spaces or Streamlit Cloud for a shareable portfolio link
- Rate limiting add SlowAPI to the README as a recommended next step - prevents abuse and runaway costs; a standard production guard for every AI API

Friday - Full Program Revision + Capstone Prep

- Full 4-week recap: LLM fundamentals → RAG → Agents → Production

Case Studies

- **Case Study 4A - AI Product Launch Strategy**
  - Healthcare startup symptom checker: compliance, clinical accuracy, user trust
  - Deliverable: responsible AI launch checklist
- **Case Study 4B - Freelance AI Consulting Engagement**
  - Restaurant chain reducing food waste using POS + inventory + weather data
  - Deliverable: 1-page proposal and project estimate

iCodeGuru × DigiTech Transformation